
Text Analysis II

Modern Approach: Analysis

...

POLI3148 Data Science in Politics and Public Administration
Dr. Chen Haohan

Review: What We Have Done

DONE

- Load text data
- Extract **textual features** from Raw Text
 - Linguistic features: tokenization, lemmatization, part-of-speech-tagging, stopword removal
 - Substantive features: Named Entities
- We call this step “**feature engineering.**”

TODO

- Question: How to use these features to generate insights about...
 - ... the whole corpus
 - ... individual documents



Generate Insights about the Whole MoFA Corpus



Overview

Objectives

- **Overview:** What is the whole collection of documents about?
- **Comparison:** What are certain parts of the documents about?

Techniques

- Calculate word frequencies → Visualize word frequencies with a word cloud
 - Fit topic modeling → Visualize topics
-

Warm-Up Exercise (5 minutes)

- Check the structure of your previous project folder
 - Raw data in the **data_raw** folder
 - Processed data in the **data_processed** folder
- Start a Colab Notebook
- Load the following MoFA datasets
 - Raw data (downloaded from the data source)
 - Linguistic features extracted from **questions** (created in previous lecture)
 - Named Entities extracted from **questions** (created in previous lecture)
- Print each of the dataset
- (If you have finished the previous steps) Find the 100 most frequent words in the whole corpus

Word Frequencies → Word Clouds

Getting General Understanding of the Corpus with **Word Frequencies**

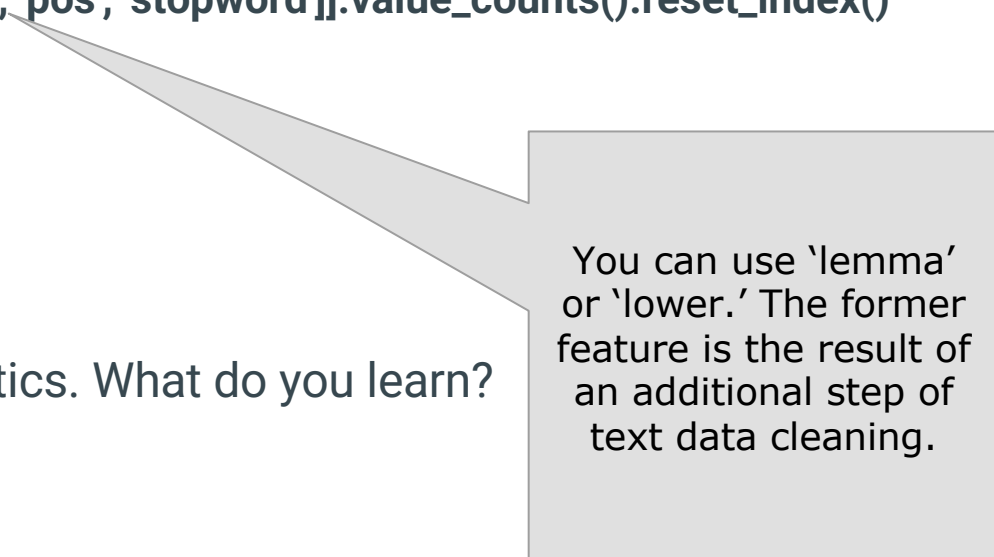
- Words are usually the basic units of documents
- Frequently-used words tell use the main content of the corpus
- However, we need pre-processing to allow us to get the substantively meaningful word. These include:
 - Removing stopwords
 - Removing punctuations
 - Lemmatization
 - Removing other irrelevant words based on part-of-speech
 - Manually remove substantively less interesting words

Here We Go

Get a table of word frequencies

```
word_frequency = text_ling.loc[:, ['lemma', 'pos', 'stopword']].value_counts().reset_index()
```

Check this table. Get summary statistics. What do you learn?



You can use 'lemma' or 'lower.' The former feature is the result of an additional step of text data cleaning.

Clean This Word Frequency Table

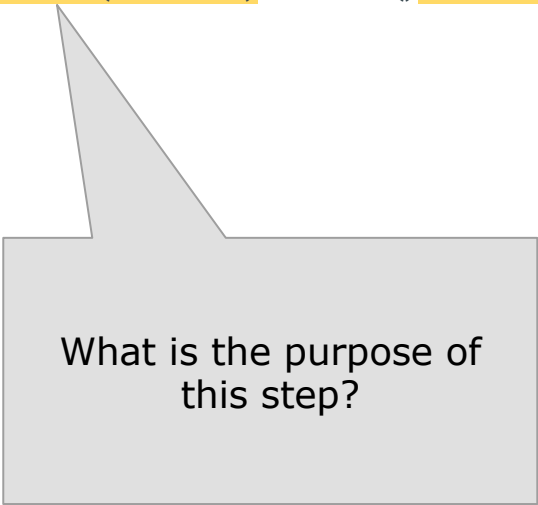
- Remove stopwords
`word_frequency = word_frequency.loc[word_frequency.stopword == False,:]`
- Remove punctuation
`word_frequency = word_frequency.loc[word_frequency.pos != "PUNCT",:]`
- Remove “less helpful” words based on domain knowledge (for example)
`word_frequency = word_frequency.loc[(word_frequency.lemma != "China") & (word_frequency.lemma != "chinese"), :]`
- Subset top 100 most frequent word
`word_frequency = word_frequency.reset_index().loc[:100, :]`



How To Make this Word Cloud: Data Preparation

Step 1: Create a dictionary indicating the word counts based on the frequency table
(*check different parts of the code to make sure you understand what it does*)

```
word_frequency_dict = word_frequency.loc[:, ['lemma', 'count']].set_index('lemma').to_dict()['count']
```



What is the purpose of this step?

Draw a Word Cloud

Step 2: Make a word cloud using the **wordcloud** library

1. Load the two libraries in your code's preamble
from wordcloud import WordCloud
import matplotlib.pyplot as plt
2. Use the following code to generate a word cloud

```
wordcloud = WordCloud()
```

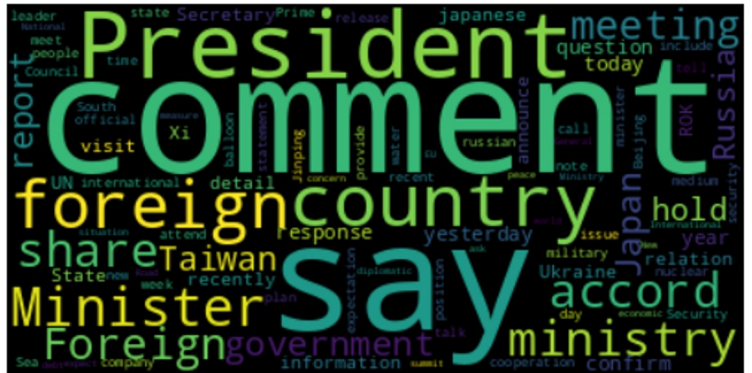
```
wordcloud.generate_from_frequencies(word_frequency_dict)
```

plt.figure()

```
plt.imshow(wordcloud, interpolation="bilinear")
```

```
plt.axis("off")
```

plt.show()



Prettify Your Word Cloud

```
wordcloud = WordCloud(width = 1000, height = 800,  
                       background_color = "white",  
                       colormap = "plasma",  
                       random_state = 1)  
  
wordcloud.generate_from_frequencies(word_frequency_dict)  
  
plt.figure()  
plt.imshow(wordcloud, interpolation="bilinear")  
plt.axis("off")  
plt.show()
```

Delete this. See what you get. Change the number. See what you get. This line guarantees that you get the SAME word cloud every time.

Want to make fancier word clouds?

Checkout the library's documentation:

https://amueller.github.io/word_cloud/auto_examples/index.html#example-gallery

But don't get too obsessed by the aesthetic designs. Afterall, the most important thing of data visualization is delivering the message. All other aesthetics are secondary.

Exercise: Make Word Clouds

1. Make word clouds to help understand -- Which ...

- organizations
- persons

... did the press care about most in the MoFA press conferences?

1. How are the questions faced by different speakers different?

Turning Text into Numbers

Concept: Bag-of-words

Turn Text to Numbers:

Count How Many Time Each Word Appears in Each Document

	she	loves	pizza	is	delicious	a	good	person	people	are	the	best
She loves pizza, pizza is delicious	1	1	2	1	1	0	0	0	0	0	0	0
She is a good person	1	0	0	1	0	1	1	1	0	0	0	0
good people are the best	0	0	0	0	0	0	1	0	1	1	1	1

Bag-of-words model

- “The bag-of-words model (BoW) is a model of text which uses a representation of text that is based on an unordered collection (a “bag”) of words.” (from [Wikipedia](#))
- A “bag-of-words” approach is a category of text analysis model that, in the model, disregard the order of words
- Our describe analysis using **Word Frequency** and **Word Cloud** implicitly uses a BoW assumption
- **Topic Modeling**, which we are discussing next, is a BoW model



Turning Text to Data for BoW models (General Approach)

- **Step 1:** Load raw text (DONE in previous steps)
- **Step 2:** Extract linguistic features from raw text (DONE in previous steps)

Raw text are turned into a long table, where each row is a word

- **Step 3:** Select and filter only the relevant linguistic features
 - Select lower case or lemmatized words
 - Remove stop words, punctuations, other less relevant words
- **Step 4:** Turn selected linguistic features into data whose formats are acceptable by the BoW model of your choice

Step 3: Select or Filter Only Relevant Linguistic Features (1)

My recipe: Remove stopwords, punctuations, numbers, and undetermined (X)

```
text_ling_s = text_ling.loc[(~text_ling.pos.isin(['X', 'PUNCT', 'NUM'])) &  
    (text_ling.stopword == False), :]
```

text_ling_s

- Check how the number of rows has changed from **text_ling** to **text_ling_s**
- Always keep the IDs

Step 3: Select or Filter Only Relevant Linguistic Features (2)

Remove extremely frequent and extremely infrequent words

- When a word is extremely frequent or extremely infrequent, it is not informative of our understanding of the corpus
- At the same time, they can slower the computation, and make the results less interpretable
- We usually want to remove extremely frequent or infrequent words before using them for BoW models

Step 3: Select or Filter Only Relevant Linguistic Features (3)

Remove extremely frequent and extremely infrequent words (con'd)

```
word_frequency = text_ling_s.lemma.value_counts().reset_index()
```

Check the distribution of frequency

```
word_frequency.reset_index().loc[:, 'count'].describe()
```

```
word_frequency.loc[:, 'count'].plot()
```

	count
count	6302.000000
mean	9.259917
std	46.960444
min	1.000000
25%	1.000000
50%	2.000000
75%	6.000000
max	2796.000000
dtype: float64	

	lemma	count
0	China	2796
1	comment	1034
2	say	817
3	chinese	653
4	report	641
...	...	...
6297	Wei	1
6298	Tang	1
6299	dynasty	1
6300	recall	1
6301	Township	1

6302 rows × 2 columns

Step 3: Select or Filter Only Relevant Linguistic Features (4)

Remove extremely frequent and extremely infrequent words (con'd)

Remove words that appear less than 10 times (use your own judgment for this number. 10 is my rule of thumb)

```
word_frequency_s = word_frequency.loc[word_frequency['count'] >= 10,:]
```

Then, keep only the selected terms in the dataset

```
text_ling_s = text_ling_s.loc[text_ling_s.lemma.isin(word_frequency_s.lemma), :]
```

Step 4: Re-format the selected linguistic feature

```
docs = text_ling_s.groupby('id')['lemma'].apply(list).reset_index()
```

	id	lemma
0	28449	[New, President, Xi, Jinping, receive, interna...
1	28450	[recently, China, announce, decision, manage, ...
2	28451	[pay, close, attention, China, communication, ...
3	28452	[russian, President, Vladimir, Putin, invite, ...
4	28453	[accord, report, President, Lula, January, rec...
...	...	...

!! PAY ATTENTION !!

A Great Leap forward

Turn text into BoW data

We want to turn our text into the following table as inputs for BoW models

	she	loves	pizza	is	delicious	a	good	person	people	are	the	best
She loves pizza, pizza is delicious	1	1	2	1	1	0	0	0	0	0	0	0
She is a good person	1	0	0	1	0	1	1	1	0	0	0	0
good people are the best	0	0	0	0	0	0	1	0	1	1	1	1

Tool: scikit-learn

<https://scikit-learn.org/stable/>

- THE most popular Python library for **Machine Learning**

So popular that you can write it on your resume!

- Can handle a variety of data, included text data
- For now, we use it to turn text into a BoW table
- *We will use it for many works in future lectures*

Transform our text into a BoW table

```
# Load the library
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# transform the document
```

```
def dummy(X):
```

```
    return X
```

```
vectorizer = CountVectorizer(tokenizer = dummy, preprocessor = dummy)
```

```
docs_bow = vectorizer.fit_transform(docs.lemma)
```

This CountVectorizer function by default takes raw text and tokenize it with a simple tokenizer. Defining the “dummy” function and setting tokenizer and preprocessor as dummy **DISABLES** it.

Why? We have already tokenized and cleaned our text.

Transform our text into a BoW table (con'd)

Take a look at our generated BoW table

`docs_bow`

`docs_bow.toarray()[ :20, :20]`

```
docs_bow.toarray()[ :20, :20]
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

`docs_vocab = vectorizer.get_feature_names_out() # Get the vocabulary`

`docs_vocab.tolist()`

`len(docs_vocab)`

```
docs_vocab.tolist()
['$',
 '%',
 '-',
 '10th',
 'APEC',
 'ASEAN',
 'AUKUS',
 'Act',
 'Advisor',
 'Affairs',
 'Afghanistan',
 'Africa',
 'African',
 'Agency',
 'Al',
 'Ambassador',
 'America',
 'American',
 'Antony',
 'April',
 'Arab',
 ...]
```

Now, you have successfully turned *text* into *numbers*!

	she	loves	pizza	is	delicious	a	good	person	people	are	the	best
She loves pizza, pizza is delicious	1	1	2	1	1	0	0	0	0	0	0	0
She is a good person	1	0	0	1	0	1	1	1	0	0	0	0
good people are the best	0	0	0	0	0	0	1	0	1	1	1	1



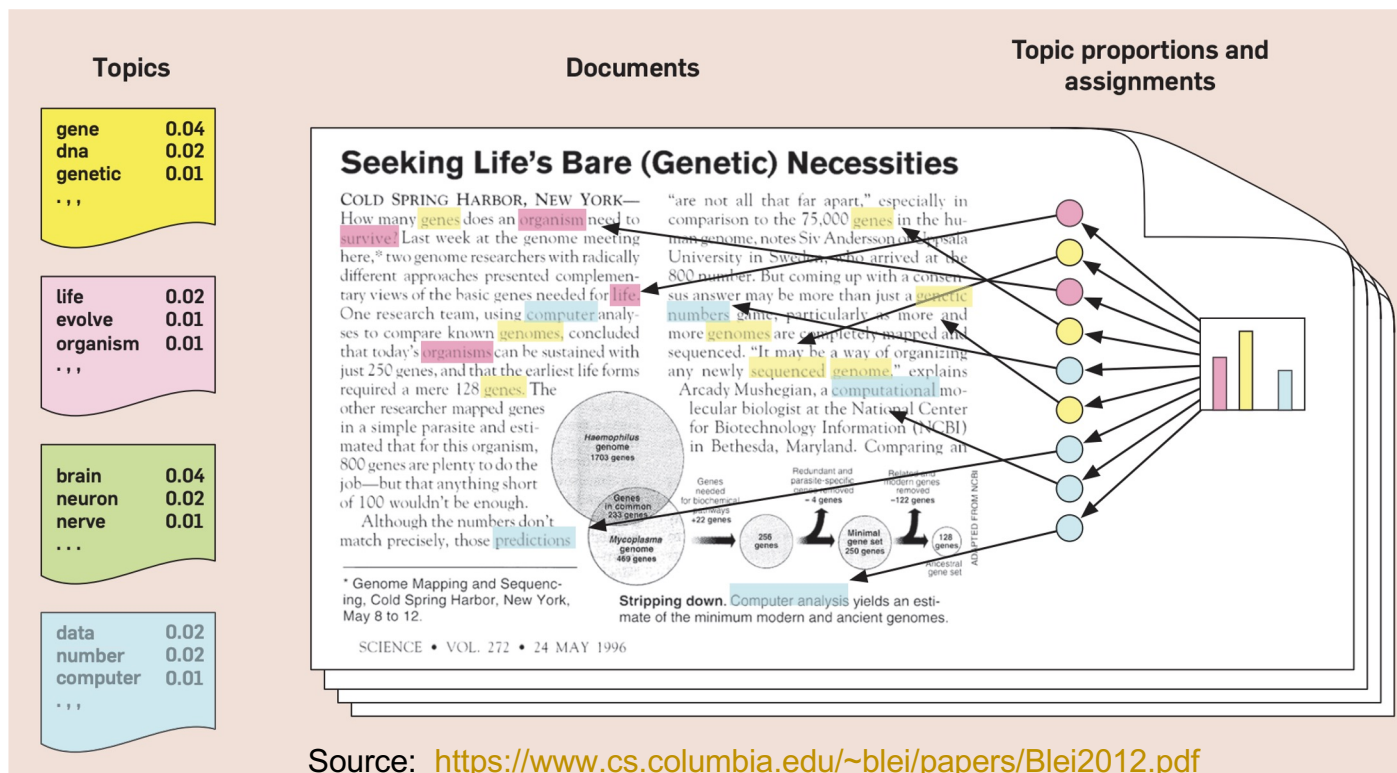
Topic Modeling



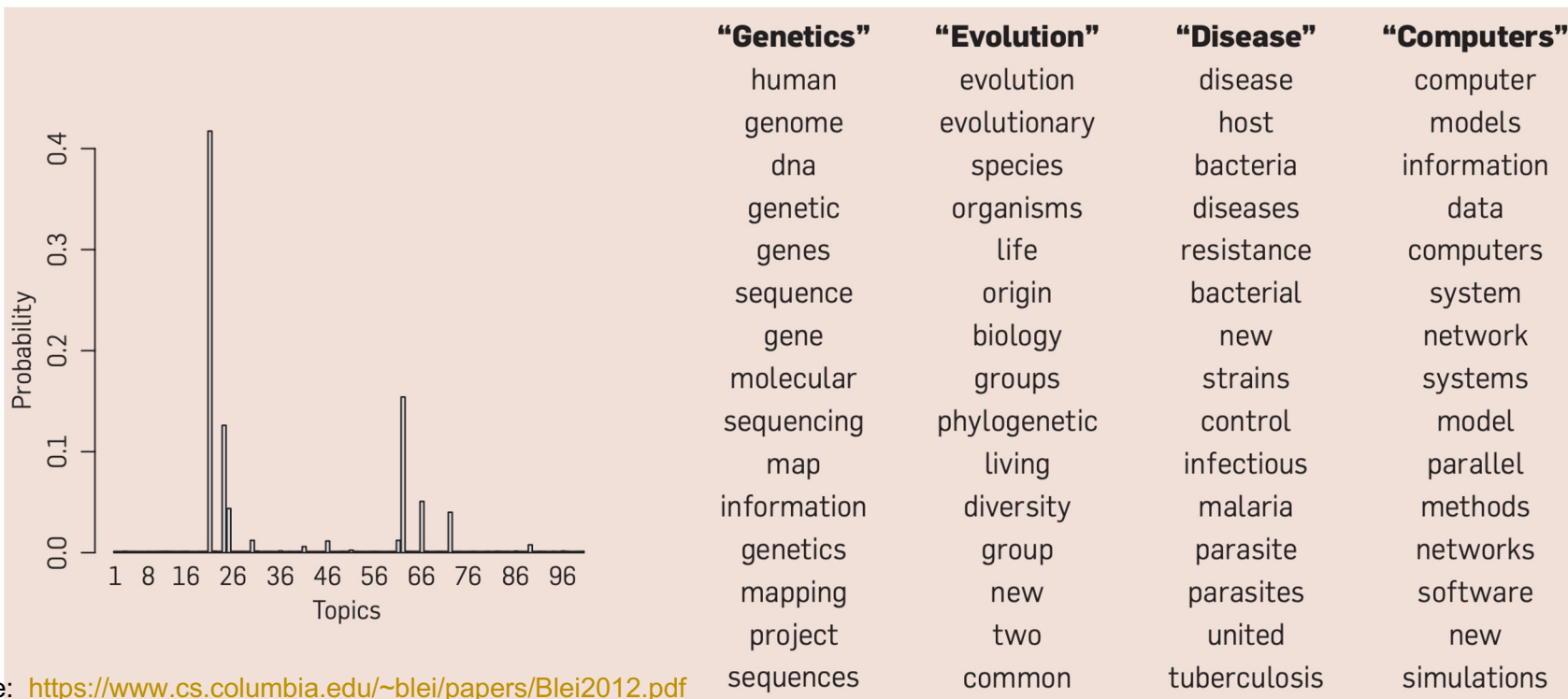
Limitations of Words

- We want to summarize a corpus into “themes”
 - As basic units of documents, words are sometimes “too small” to tell characterize these themes
 - We need a more powerful model that can effectively summarize documents into “themes”
 - → TOPIC MODELING!
-

Topic Modeling as “Highlighting”: Illustration 1



Topic Modeling as “Highlighting”: Illustration 1



Source: <https://www.cs.columbia.edu/~blei/papers/Blei2012.pdf>

Topic Modeling as “Highlighting”: Illustration 2

“Arts”	“Budgets”	“Children”	“Education”
NEW	MILLION	CHILDREN	SCHOOL
FILM	TAX	WOMEN	STUDENTS
SHOW	PROGRAM	PEOPLE	SCHOOLS
MUSIC	BUDGET	CHILD	EDUCATION
MOVIE	BILLION	YEARS	TEACHERS
PLAY	FEDERAL	FAMILIES	HIGH
MUSICAL	YEAR	WORK	PUBLIC
BEST	SPENDING	PARENTS	TEACHER
ACTOR	NEW	SAYS	BENNETT
FIRST	STATE	FAMILY	MANIGAT
YORK	PLAN	WELFARE	NAMPHY
OPERA	MONEY	MEN	STATE
THEATER	PROGRAMS	PERCENT	PRESIDENT
ACTRESS	GOVERNMENT	CARE	ELEMENTARY
LOVE	CONGRESS	LIFE	HAITI

Source:

<https://dl.acm.org/doi/pdf/10.5555/944919.944937>

The William Randolph Hearst Foundation will give \$1.25 million to Lincoln Center, Metropolitan Opera Co., New York Philharmonic and Juilliard School. “Our board felt that we had a real opportunity to make a mark on the future of the performing arts with these grants an act every bit as important as our traditional areas of support in health, medical research, education and the social services,” Hearst Foundation President Randolph A. Hearst said Monday in announcing the grants. Lincoln Center’s share will be \$200,000 for its new building, which will house young artists and provide new public facilities. The Metropolitan Opera Co. and New York Philharmonic will receive \$400,000 each. The Juilliard School, where music and the performing arts are taught, will get \$250,000. The Hearst Foundation, a leading supporter of the Lincoln Center Consolidated Corporate Fund, will make its usual annual \$100,000 donation, too.

Understanding Topic Modeling: Behavioral Assumption

The assumed Behavioral Model of topic modeling:

When an “author” *generate* a document (e.g., MoFA question, answer, tweet), they

- Have a **vocabulary** in mind
- Have a set of “**topics**” in mind
- Know how the **vocabulary** is associated with the “**topics**” ⇒
- Pick the proportions that each “**topic**” should take in the document ⇒
- Pick **words** according to the proportions of topics
- Generate the **document**, which is a *bag of words*

IMPORTANT: You need to understand this to interpret topic modeling correctly

Understanding Topic Modeling: Parameters

Topic Modeling is a model for text analysis that generates insights under the above assumption. Here is its required inputs and outputs

Needed Inputs

- Tokenized documents
- (IMPORTANT) Number of topics (K)

Outputs

- A set of topics, each represented by its *most relevant* words
- (for each document) a K -dimensional vector indicating its relevance to each of the K topics

Fit Topic Modeling

Load the function for topic modeling

```
from sklearn.decomposition import LatentDirichletAllocation
```

Initiate the model. **n_components** controls the number of topics

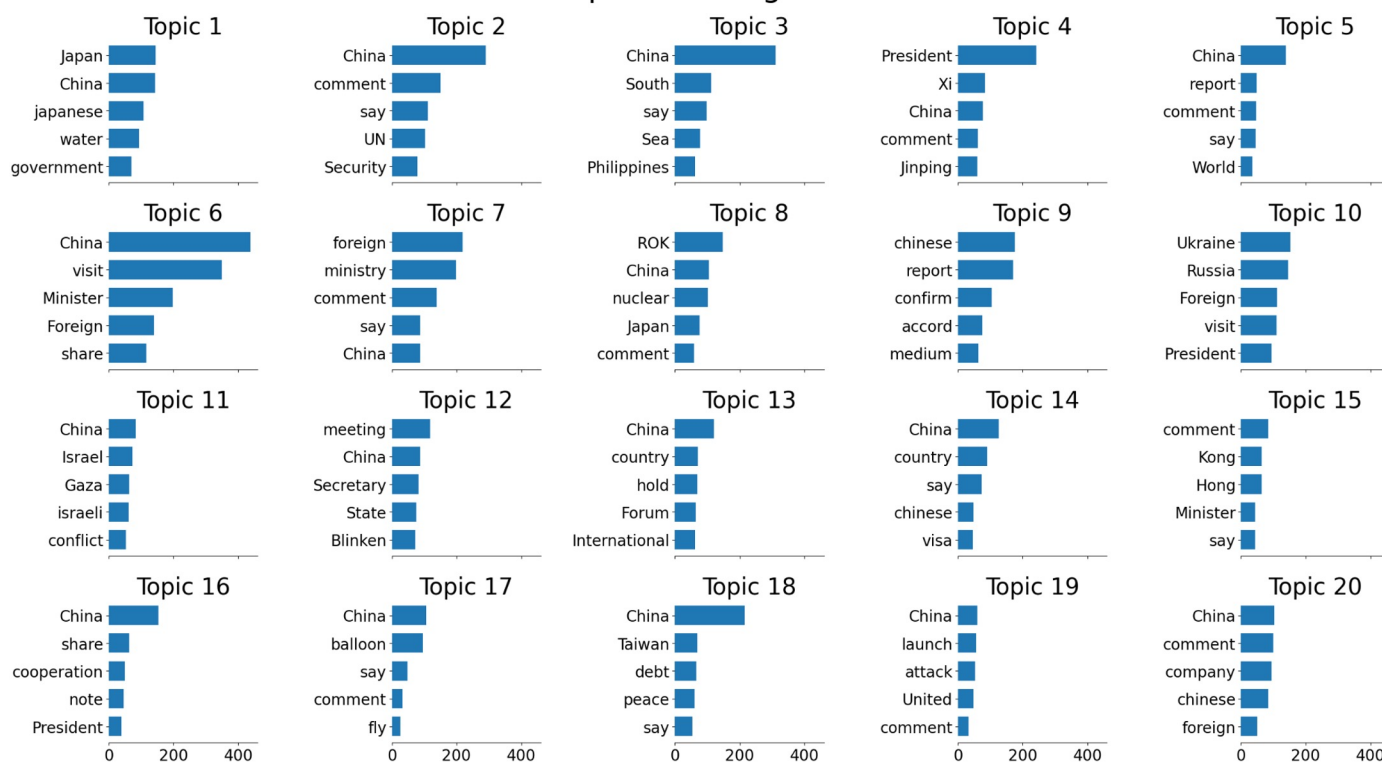
```
model_lda = LatentDirichletAllocation(n_components=20, random_state=0)
```

Fit the model

```
model_lda.fit(docs_bow)
```

Visualize Results of Topic Modeling

Topic Modeling Results



```
def plot_topic_top_words(model, feature_names, n_top_words, title, fig_n_row, fig_n_col):
```

```
    fig, axes = plt.subplots(fig_n_row, fig_n_col, figsize=(30, 15), sharex=True)
```

```
    axes = axes.flatten()
```

```
    for topic_idx, topic in enumerate(model.components_):
```

```
        top_features_ind = topic.argsort()[-n_top_words:]
```

```
        top_features = feature_names[top_features_ind]
```

```
        weights = topic[top_features_ind]
```

```
        ax = axes[topic_idx]
```

```
        ax.barh(top_features, weights, height=0.7)
```

```
        ax.set_title(f"Topic {topic_idx + 1}", fontdict={"fontsize": 30})
```

```
        ax.tick_params(axis="both", which="major", labelsize=20)
```

```
        for i in "top right left".split():
```

```
            ax.spines[i].set_visible(False)
```

```
    fig.suptitle(title, fontsize=40)
```

```
plt.subplots_adjust(top=0.90, bottom=0.05, wspace=0.90, hspace=0.3)
```

```
plt.show()
```

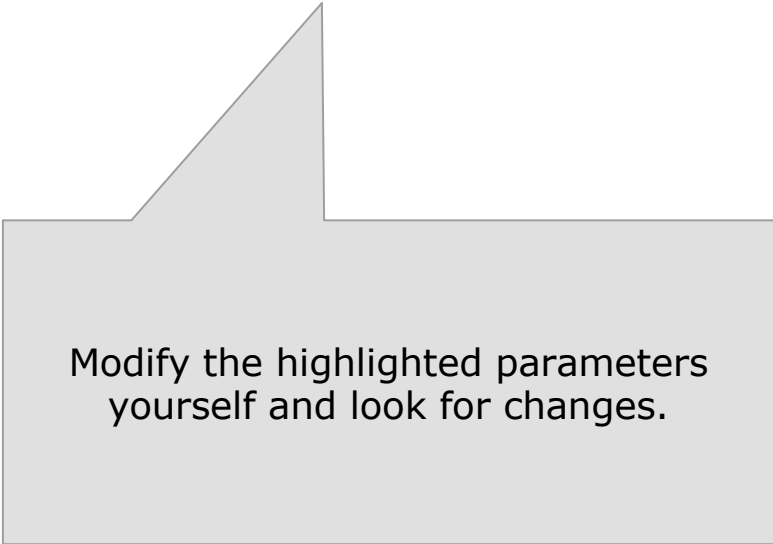
Define this function to generate a quick plot showing topics and top words associated with them.

Check for yourself what each part of it does.

[Source of the function](#). Minor modification by Dr. Haohan Chen

Call the function to visualize topics

```
plot_topic_top_words(model_lda, docs_vocab, 5, "Topic Modeling Results", 4, 5)
```



Modify the highlighted parameters
yourself and look for changes.

Improving Topic Modeling Results

Improve topic modeling to make it more interpretable

- Observe and remove words that appear to make topics uninformative
- Change the number of topics?
- * Concatenate named entities

Improve visualization

- Name the topics
- Names of topics on the data visualization
- Keep only topics that you can name?

Apply Text Analysis Skills to a Different Language?

Skills for Text Analysis is Generalizable

Languages look different. But computational approach to them are transferable

- Text analysis methods are usually developed and tested with reference to one or a few language(s) (most frequently, **English**, and somewhat frequently, **Chinese**)
- When you find a method useful for analyzing texts in one language, you are likely to find it useful for other languages
- Algorithms for feature extraction (linguistic, substantive) are more often language-specific. But large toolkits provides workflow applicable across languages (e.g., spaCy)
- Large Language Models, as the latest development, show much improved multi-lingual capacity -- but the cost is they consume more computational resources

Demo: Apply spaCy to Chinese

```
spacy.cli.download("zh_core_web_sm")
```

```
nlp = spacy.load("zh_core_web_sm")
```

Caveat: Linguistic features that we can generate for English need not be available for other language. For example, there is no lemmatization for Chinese.

Read the documentation. But the documentation is not always informative. Poke around. Try.

TRAINED PIPELINES

Catalan
Chinese
Croatian
Danish
Dutch
English
Finnish
French
German
Greek
Italian
Japanese
Korean
Lithuanian
Macedonian
Multi-language
Norwegian Bokmål
Polish
Portuguese
Romanian
Russian
Slovenian
Spanish
Swedish
Ukrainian

Running Example: MoFA data (English + Chinese)

- We merged the Chinese and English versions of a part of the questions
(*special thanks to our TA, Mr. Liao Li*)
- Perform the same analyses for all steps that follow
 - **Extracting linguistic features**
 - **Extracting named entities**
 - **Visualize word frequencies**
 - Turn text into numbers using a BoW approach
 - Make word clouds
 - Perform Topic Modeling

Let's try a few steps together!

Including Chinese in a Word Cloud

- Problem: Visualization containing Chinese may be incorrectly displayed:



- Reason: Colab does not have built-in Chinese fonts
- Solution:
 - Download a Chinese font (e.g., [SimHei](#), feel free to try [others](#) but beware of readability)
 - Tell the *matplotlib* library to use the font

Including Chinese in a Word Cloud (con'd)

ATTENTION: Change the path to locations where you store your fonts.

```
PATH_FONT = os.path.join(PROJECT_FOLDER, 'data_chinese_font/simhei/SimHei.ttf')
```

```
wordcloud = WordCloud(font_path = PATH_FONT)
```

```
wordcloud.generate_from_frequencies(word_frequency_dict)
```

```
plt.figure()
```

```
plt.imshow(wordcloud, interpolation="bilinear")
```

```
plt.axis("off")
```

```
plt.show()
```



Including Chinese in other *Seaborn* and *Matplotlib* Outputs

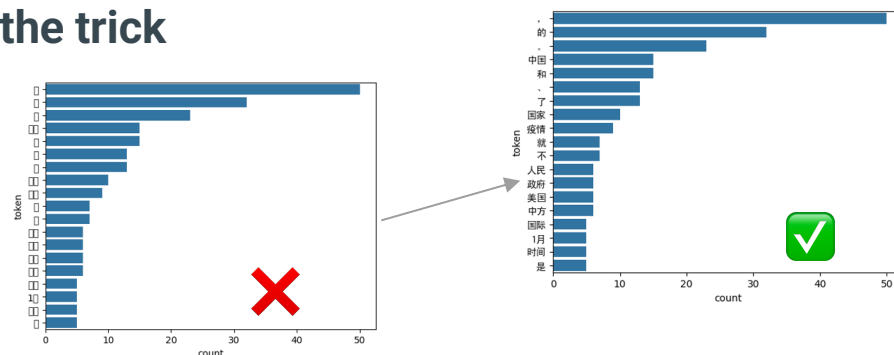
```
word_frequency_s2 = word_frequency_s[word_frequency_s['count'] >= 5]
```

```
font_cn = matplotlib.font_manager.FontProperties(fname = PATH_FONT)
```

```
sns.barplot(word_frequency_s2, y = "token", x = "count")
```

```
plt.yticks(fontproperties = font_cn) # This is the trick
```

```
plt.show()
```



That's it.

**That is the end of the
beginning of your text
analysis journey!**

What we have covered and not

The ABC of text analysis

- A selected set of tools that I found rigorous
- Basic methods that I found helpful
- Intuitions of some “fundamentals” underlying these methods

What we have not covered

- Technical details (mathematical proofs, engineering)
- A lot of other tools available
- *Tools come and go*. What is important is:
 - You understand the logic of the “fundamentals” (e.g., the BoW approach)
 - You acquire the mindset that helps you understand the “fundamentals”

Note

There are other tools that can perform the text analysis we have done in the past two sessions

- For textual feature extraction: [NLTK](#)
- For topic modeling: [Gensim](#)

I choose a small set of tools to demonstrate how things work and how to do it in an efficient way. To share generalizable knowledge under time constraint, I am very selective, and cover only a few.

Feel free to explore other tools. But beware of the quality and suitability for your tasks of interest.



Text Analysis with Large Language Models

A Postmodern Approach

Large Language Models

... But different from how the most of you have used them

- You have probably used large language models in chatbots
- You have probably used commercial Large Language Models (e.g., ChatGPT)

What you will learn: Large Language Models can be integrated into your text analysis workflow:

- Use open-sourced Large Language Models
- Interact with models using code
- Store the models locally
- Use Google Colab's computational device

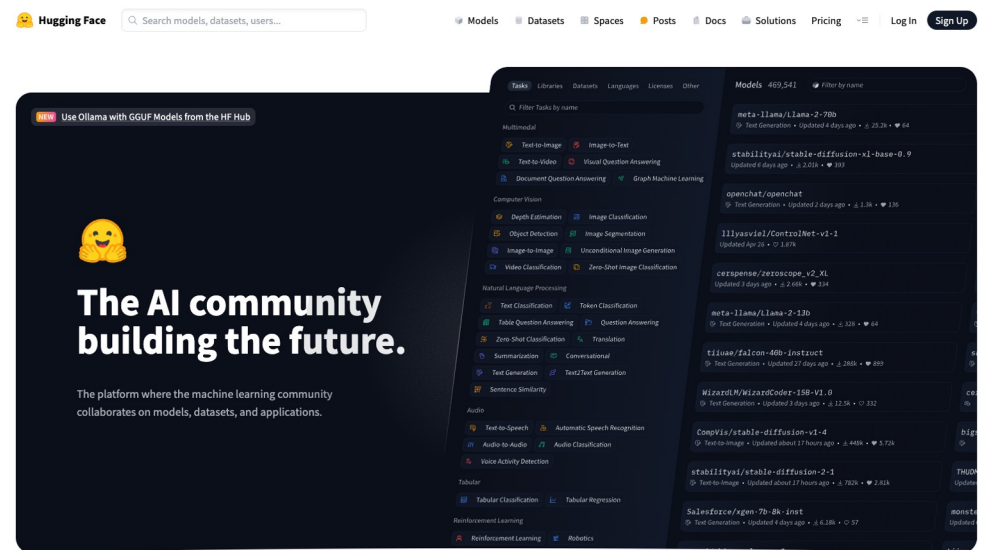
Prerequisite: HuggingFace

<https://huggingface.co/>

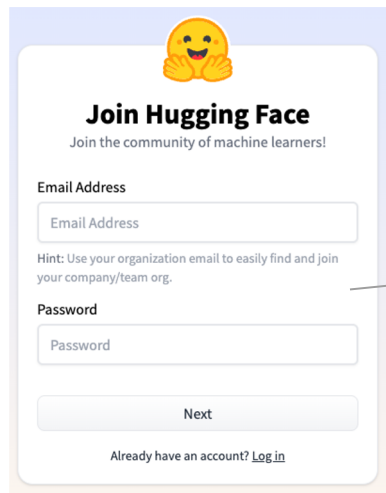
The hub of open-sourced AI models (including large language models)

Register for free

Then get an *access token*



Register for HuggingFace



Join Hugging Face
Join the community of machine learners!

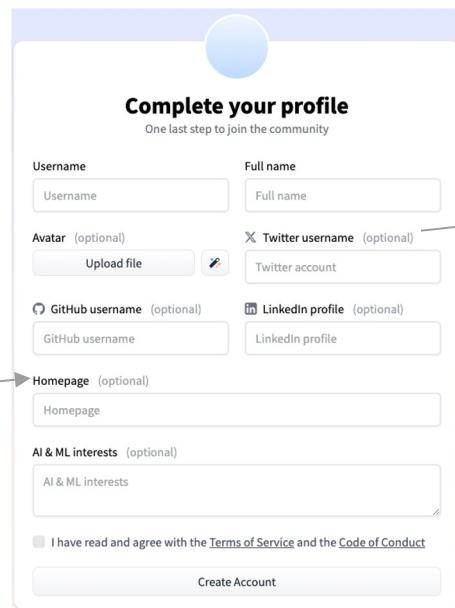
Email Address
Email Address

Hint: Use your organization email to easily find and join your company/team org.

Password
Password

Next

Already have an account? [Log in](#)



Complete your profile
One last step to join the community

Username
Username

Full name
Full name

Avatar (optional)
Upload file

Twitter username (optional)
Twitter account

GitHub username (optional)
GitHub username

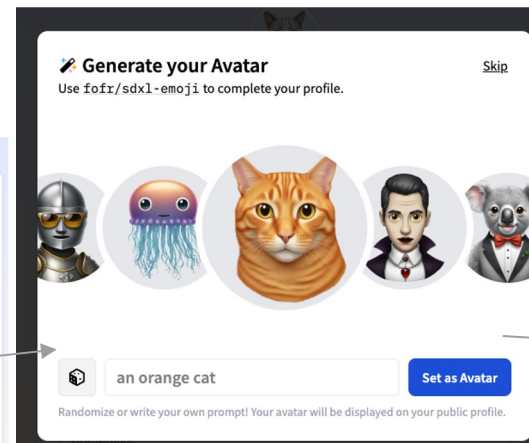
LinkedIn profile (optional)
LinkedIn profile

Homepage (optional)
Homepage

AI & ML interests (optional)
AI & ML interests

☐ I have read and agree with the [Terms of Service](#) and the [Code of Conduct](#)

Create Account



Generate your Avatar
Use `fofr/sdxl-emoji` to complete your profile.

Skip

an orange cat

Set as Avatar

Randomize or write your own prompt! Your avatar will be displayed on your public profile.

Let's confirm you are human

Complete the security check before continuing. This step verifies that you are not a bot, which helps to protect your account and prevent spam.

Begin >

English

[Hugging Face] Click this link to confirm your email address [Inbox x](#)

huggingface <website@huggingface.co>
to me

IMPORTANT: Check your email.

Confirm your email address before going to next step.



Confirm your email address by clicking on this link:

Get an Access Token

YOU SHOULD FIRST CONFIRM YOUR EMAIL ADDRESS, otherwise HuggingFace won't let you do this step.

The screenshot shows the Hugging Face 'Create new Access Token' interface. On the left is a sidebar with navigation links: Solutions, Pricing, Profile (HChenH), Notifications, Inbox (0), New Model, New Dataset, New Space, New Collection, Create organization, Settings, Access Tokens (highlighted), Billing, and Sign Out. The main content area is titled 'Access Tokens' and shows 'User Access Tokens' with a 'Create new token' button. Below this is a message: 'You have no Access Token'. To the right, the 'Create new Access Token' modal is open. It shows 'Token type' as 'Fine-grained' with 'Read' selected. A note states: 'This cannot be changed after token creation.' The 'Token name' field contains 'LearnDataScience' with an annotation: 'Doesn't matter. Whichever name you prefer'. Below this is a description: 'This token has read-only access to all your and your orgs resources and can make comment on discussions.' and a 'Create token' button. At the bottom, a 'Save your Access Token' modal is shown, displaying the token value 'hf_CtTziUeduivZTyYU0SEfIyIzVMVGXUqbt' with a 'Copy' button. Below the token value is a table with 'Name' (LearnDataScience) and 'Permissions' (READ). A 'Done' button is at the bottom right. Two yellow text boxes with arrows point to the 'Create token' and 'Done' buttons respectively.

Access Tokens

User Access Tokens

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions.

- Do not share your Access Tokens with anyone; we regularly check for leaked Access Tokens and remove them immediately.

You have no Access Token

Create new Access Token

Token type

Fine-grained **Read** Write

This cannot be changed after token creation.

Token name

LearnDataScience

Doesn't matter. Whichever name you prefer

This token has read-only access to all your and your orgs resources and can make comment on discussions.

Create token

Save your Access Token

Save your token value somewhere safe. You will not be able to see it again after you close this modal. If you lose it, you'll have to create a new one.

hf_CtTziUeduivZTyYU0SEfIyIzVMVGXUqbt

Copy

Name	Permissions
LearnDataScience	READ

Done

Hit "Done." No need to remember or copy the token now.

When you need to use it, you can "invalidate and refresh" to obtain a new token.



Running Large Language Models on Google Colab

Use this guide: https://github.com/casualcomputer/llm_google_colab

Let's work on it together from the source.

This is a demonstration of how to learn from online resources and apply it to your own work.

If you can't follow, feel free to use the local copy in the demo folder.

